AbyssCore

Building a Cloud-Native Dungeon Crawler on Kubernetes

Bas Smit

Fontys University of Applied Sciences

Fontys
University of Applied Sciences

AbyssCore

Building a Cloud-Native Dungeon Crawler on Kubernetes

by

Bas Smit

Personal Project — Semester 7

Fontys University of Applied Sciences, Faculty of IT Eindhoven

Project duration:	February 2026 – June 2026
Mentor:	Ederveen, Martin M.W.
Course Head:	Meulensteen, Tom T.
Faculty:	Faculty of IT, Eindhoven

Fontys HOGESCHOOL ICT

Preface

This document describes the design, development, and deployment of AbyssCore — a cloud-native dungeon crawler game built as a personal project during semester 7 at Fontys University of Applied Sciences. The original project plan focused on AI-powered Kubernetes monitoring with Cilium, but the scope shifted to something more hands-on: actually building and running a real distributed application on Kubernetes from scratch, and learning infrastructure engineering by doing it for real rather than in theory.

The result is a working game backend with a microservice architecture, a GitOps-managed Kubernetes cluster, full observability through Prometheus and Grafana, and public exposure via Cloudflare Tunnel — all running on Fontys-managed infrastructure.

Abstract

AbyssCore is a cloud-native dungeon crawler game built on a microservice architecture using Encore.go for the backend, a GraphQL gateway in Go, and a Next.js frontend. The entire stack runs on a Fontys-managed Kubernetes cluster with GitOps-driven deployments via ArgoCD, full observability through kube-prometheus-stack, and public access through Cloudflare Tunnel.

This document covers the full development journey: architecture decisions, infrastructure setup, the constraints imposed by the Fontys network environment, the CI/CD pipeline, and the observability stack. It also discusses the current state of scalability and what would need to change to run AbyssCore in a production environment.

Keywords: Kubernetes, GitOps, ArgoCD, Encore.go, GraphQL, Cloudflare Tunnel, Prometheus, Grafana, microservices

Contents

Preface **iii**

Abstract **iv**

1 Introduction **1**
1.1 Background and Motivation . . . 1
1.2 What Changed from the Project Plan . . . 1
1.3 Project Objectives . . . 1
1.4 Document Structure . . . 1

2 System Architecture **3**
2.1 Overview . . . 3
2.2 Technology Stack . . . 3
2.2.1 Next.js Frontend . . . 3
2.2.2 Encore.go Backend . . . 3
2.2.3 GraphQL Gateway . . . 4
2.2.4 Keycloak . . . 4
2.2.5 PostgreSQL and RabbitMQ . . . 4
2.3 Architecture Decisions . . . 4
2.3.1 One Binary for All Backend Services . . . 4
2.3.2 Database Per Service (Same Instance) . . . 4
2.3.3 GraphQL Over REST for the Frontend . . . 5
2.3.4 Cloudflare Tunnel for External Access . . . 5

3 Infrastructure and Deployment **6**
3.1 The Cluster . . . 6
3.2 GitOps with ArgoCD . . . 6
3.2.1 Self-Healing . . . 6
3.3 Image Mirroring and CI/CD . . . 7
3.3.1 The Problem . . . 7
3.3.2 GitHub Actions Pipeline . . . 7
3.3.3 Environment Variables . . . 7
3.4 Public Access via Cloudflare Tunnel . . . 7
3.5 Resource Management . . . 8
3.6 Test and Production Environments . . . 8
3.6.1 The Plan . . . 8
3.6.2 Image Tagging . . . 8
3.6.3 Kustomize Overlays . . . 9
3.6.4 ArgoCD Applications . . . 9
3.6.5 Cloudflare Hostnames . . . 9
3.6.6 Outcome . . . 9

4 Observability **10**
4.1 The Monitoring Stack . . . 10

4.2 What Gets Scraped 10
4.3 Grafana 11
4.4 What's Missing 11

5 Scalability 12
5.1 Current State 12
5.2 Load Balancing 12
5.3 Horizontal Pod Autoscaler (HPA) 12
5.4 Vertical Pod Autoscaler (VPA) 13
5.5 Cluster Autoscaler 13
5.6 Scaling Recommendations for Production 13
5.7 Planned: Authentication for All Services 14

References 16

A Reflection and Future Work 17
AA What Went Well 17
AB What Was Harder Than Expected 17
AC What Changed from the Project Plan 18
AD Future Work 18

B Appendix A — Kubernetes Manifest Overview 19

C Appendix B — ArgoCD Application Definitions 20

D Appendix C — CI/CD Pipeline Summary 21

Introduction 1

1.1 Background and Motivation

Kubernetes has become the default way to run containerised applications at scale. But most tutorials and courses skip straight to the theory and never show you what it actually looks like to take a real application from zero to production on a managed cluster. There are a lot of moving parts — container registries, ingress, secrets, DNS, observability — and most of the pain only shows up when you're actually doing it.

AbyssCore started as a way to fix that gap. The idea was simple: build a real application, deploy it on a real Kubernetes cluster, and figure out every problem along the way. No kind cluster on a laptop, no managed cloud with one-click ingress. A real Fontys-managed cluster, behind a corporate TLS-intercepting proxy, with all the constraints that come with it.

The application itself is a dungeon crawler game. Players explore procedurally generated dungeons, fight enemies, manage inventory, and compete on a leaderboard. It's complex enough to justify a microservice architecture — multiple independent services, a message broker, authentication, a database per service — but focused enough to actually finish.

1.2 What Changed from the Project Plan

The original project plan described an AI-powered monitoring system built on top of Cilium and eBPF, investigating anomaly detection in Kubernetes infrastructure. That scope was ambitious, and in hindsight, too infrastructure-focused without a real application underneath it to monitor.

The pivot to AbyssCore made the project more grounded. Instead of simulating a monitored system, there's an actual application to run, scale, and observe. The infrastructure challenges turned out to be just as interesting as originally planned — Fontys's network environment introduced real constraints around image pulling, DNS, and TLS that required genuine engineering to solve. The observability stack (Prometheus, Grafana, ServiceMonitors, ArgoCD) ended up being a significant part of the project too, just driven by a real need rather than theory.

1.3 Project Objectives

The main objectives of this project were:

- Build a working dungeon crawler game with a proper microservice backend
- Deploy the full stack on a Fontys-managed Kubernetes cluster using GitOps principles
- Set up production-grade observability with Prometheus and Grafana
- Make the application publicly accessible without relying on NodePort or cloud LoadBalancers
- Document the architecture, the decisions, and the problems that came up along the way

1.4 Document Structure

This document is structured as follows:

- **Chapter 2** covers the system architecture — what was built and why it's structured the way it is.

- **Chapter 3** describes the infrastructure and deployment setup, including the Kubernetes cluster, ArgoCD, and CI/CD pipeline.
- **Chapter 4** covers observability — the monitoring stack, what's being measured, and what's still missing.
- **Chapter 5** discusses scalability — how the current setup handles load and what would need to change for production.
- **Chapter 6** reflects on what went well, what didn't, and what comes next.

System Architecture 2

2.1 Overview

AbyssCore is built as a set of independent services that communicate over HTTP and a message queue. Each service owns its own database, there's no shared data layer between them. A GraphQL gateway sits in front of everything and gives the frontend a single endpoint to talk to. Authentication is handled by Keycloak, which runs as a separate service in the cluster.

The high-level structure looks like this:

```
Browser
  └── Next.js Frontend (port 3000)
        └── GraphQL Gateway (port 4001)
              ├── Backend – Combat service (port 4000)
              ├── Backend – Dungeon service (port 4000)
              ├── Backend – Inventory service (port 4000)
              ├── Backend – Deck service (port 4000)
              ├── Backend – Map service (port 4000)
              ├── Backend – Game service (port 4000)
              └── Backend – Leaderboard service (port 4000)
                    └── RabbitMQ (message broker)
Keycloak (auth, port 8080)
PostgreSQL (one database per service)
```

All external traffic arrives through Cloudflare Tunnel and never touches a NodePort or LoadBalancer. Inside the cluster, services communicate directly over Kubernetes DNS.

2.2 Technology Stack

2.2.1 Next.js Frontend

The frontend is a standard Next.js application. It talks to the GraphQL gateway for all game data and to Keycloak for authentication. NextAuth.js handles the OAuth2/OIDC flow — it exchanges the Keycloak token for a session, and then passes a JWT to Apollo Client on every GraphQL request.

The frontend is stateless and runs as a single Kubernetes Deployment. It doesn't store anything itself; everything is fetched from the backend.

2.2.2 Encore.go Backend

The backend is built with Encore.go, a framework that generates boilerplate for microservices in Go. Each game domain — combat, dungeon, inventory, deck, map, game, leaderboard — is a separate Encore service. Encore handles routing, database migrations, and inter-service calls through typed APIs, which keeps the code clean and avoids a lot of the repetitive stuff you'd normally write.

All services share a single compiled binary that Encore produces, which gets packaged into one Docker image. That image is deployed as one Kubernetes Deployment. Encore routes requests internally to the right service based on the URL path.

Each service has its own PostgreSQL database. They all run on the same Postgres instance in the cluster but in separate databases, so there's no schema sharing. This is a pragmatic compromise — fully separate Postgres instances per service would be more correct architecturally, but the Fontys cluster has resource limits and a single instance is much easier to manage.

2.2.3 GraphQL Gateway

The gateway is a hand-written Go service using Gin as the HTTP router. It exposes a single GraphQL endpoint and translates GraphQL queries into HTTP calls to the Encore backend. It uses gqlgen for schema-first GraphQL code generation.

The reason for a separate gateway instead of just calling the backend directly from the frontend is flexibility. The frontend speaks GraphQL, the backend speaks REST. The gateway handles that translation, and it also means the frontend doesn't need to know anything about the internal service structure.

2.2.4 Keycloak

Keycloak handles all authentication. There's one realm (`abysscore`) and one client (`abysscore-frontend`). Players log in through the standard OIDC flow — Keycloak issues a JWT, NextAuth stores it in the session, and that JWT gets sent with every request to the gateway.

The gateway validates the token on every request before forwarding anything to the backend. The backend itself doesn't do auth checks — it trusts that the gateway has already verified the caller.

2.2.5 PostgreSQL and RabbitMQ

PostgreSQL runs as a single StatefulSet in the cluster. There are seven databases — one per backend service. Migrations run automatically when the backend starts, handled by Encore.

RabbitMQ is used for async communication between services. The main use case is leaderboard updates — when a game ends, the game service publishes an event and the leaderboard service picks it up and updates the scores. This decouples the two services so neither has to wait for the other.

2.3 Architecture Decisions

2.3.1 One Binary for All Backend Services

Encore compiles all services into a single binary. The alternative would be separate deployments per service, which would be more "correct" microservice architecture but adds a lot of operational complexity — separate Deployments, separate health checks, separate resource quotas. For a project at this scale, one binary is the pragmatic choice. The services are still logically separate and don't share code, just a process.

2.3.2 Database Per Service (Same Instance)

Each service has its own database, which enforces service boundaries and prevents accidental coupling through shared tables. But they all run on the same Postgres instance. This is a deliberate trade-off: it gives the benefits of logical separation without the cost of running and maintaining seven separate database instances on a resource-constrained cluster.

2.3.3 GraphQL Over REST for the Frontend

GraphQL lets the frontend request exactly the data it needs in a single round trip. For a game with multiple services and data types, this is a real benefit — the frontend can compose queries across dungeon state, inventory, and player stats in one request rather than making three separate REST calls and stitching the results together.

2.3.4 Cloudflare Tunnel for External Access

The Fontys cluster nodes are on a private network that isn't reachable from the internet. The normal Kubernetes approach — NodePort or LoadBalancer — doesn't work here. Cloudflare Tunnel solves this cleanly: a `cloudflared` pod inside the cluster opens an outbound connection to Cloudflare's edge, and Cloudflare routes external traffic back through that tunnel. No inbound firewall rules, no NodePort exposure, no cloud LoadBalancer needed.

Infrastructure and Deployment 3

3.1 The Cluster

AbyssCore runs on a Kubernetes cluster managed by Fontys. This means no access to the control plane, no ability to install cluster-wide components without working around restrictions, and — most importantly — a TLS-intercepting proxy that intercepts all outbound HTTPS traffic.

That last point caused the most friction. Kubernetes pulls container images over HTTPS. When the proxy intercepts those connections, it replaces the TLS certificate with its own. Docker Hub, GitHub Container Registry, and Quay all use certificate pinning or expect specific certificates — so every image pull from an external registry fails with a TLS error.

The fix: mirror every image to GitHub Container Registry under the `bassmit12` organisation, and configure Kubernetes to pull from there using a `ghcr-secret` image pull secret. Any new image used in the cluster needs to be mirrored first. It's extra work, but it's a one-time cost per image and after that it's reliable.

3.2 GitOps with ArgoCD

All Kubernetes manifests live in the AbyssCore GitHub repository under `infra/`. ArgoCD watches the repository and automatically applies changes when new commits land on `main`. There's no manual `kubectl apply` in the normal workflow — you push to git and ArgoCD handles the rest.

Three ArgoCD Applications manage the full stack:

- **abysscore** — the game itself: frontend, backend, gateway, Keycloak, RabbitMQ, PostgreSQL, Cloudflare Tunnel. Watches `infra/k8s/`.
- **monitoring** — the kube-prometheus-stack Helm chart. Deploys Prometheus, Alertmanager, Grafana, node-exporter, and kube-state-metrics. Sourced directly from the prometheus-community Helm repo.
- **monitoring-extras** — ServiceMonitors and other monitoring add-ons that live in git. Watches `infra/k8s/monitoring/`.

ArgoCD runs in its own namespace and is exposed publicly at `argocd.bassmit.dev` via Cloudflare Tunnel. It runs in insecure mode (no internal TLS) because the Cloudflare Tunnel handles TLS termination at the edge.

3.2.1 Self-Healing

One of the big benefits of GitOps is that the cluster will always converge back to what's in git. If someone manually edits a Deployment, ArgoCD will overwrite it on the next sync. This makes accidental drift impossible to sustain — the git repository is always the source of truth.

3.3 Image Mirroring and CI/CD

3.3.1 The Problem

As described above, the Fontys proxy blocks external image pulls. But there's a second constraint: building and pushing images in CI also needs registry access. The solution is to use GitHub Actions to build images and push them to GHCR, then have Kubernetes pull from GHCR using a secret.

3.3.2 GitHub Actions Pipeline

The CI pipeline runs on every push to `main`. It has three parallel jobs:

- **Build Backend** — runs `encore build docker` to compile the Encore application and push `ghcr.io/bassmit12/abysscore-backend:<sha>` and `:latest`
- **Build Gateway** — builds the Go gateway with Docker Buildx and pushes `ghcr.io/bassmit12/abysscore-gateway:<sha>` and `:latest`
- **Build Frontend** — builds the Next.js app with Docker Buildx and pushes `ghcr.io/bassmit12/abysscore-frontend:<sha>` and `:latest`

After the three build jobs finish, an `update-manifests` job patches the image tags in `infra/k8s/*.yaml` from the previous SHA to the new one and commits the change back to `main`. ArgoCD detects the manifest update and rolls out all three deployments automatically.

One important detail: the manifest-update commit must not trigger a new build. Without a guard, the bot commit lands on `main`, triggers the workflow again, which pushes new images, updates the manifest, commits again — an infinite loop. The fix is `paths-ignore: infra/k8s/**` on the workflow trigger. The CI only fires on changes to actual source code, not on manifest-only commits.

Authentication uses a `GHCR_TOKEN` secret stored in GitHub Actions. The `GITHUB_TOKEN` built into Actions fails with a 403 for new packages, so a personal access token with `write:packages` scope is required.

3.3.3 Environment Variables

The frontend needs to know the public URLs for the GraphQL gateway and Keycloak at build time, since Next.js bakes `NEXT_PUBLIC_*` variables into the static bundle. These are passed as build arguments in the GitHub Actions workflow and come from repository secrets:

- `NEXT_PUBLIC_GRAPHQL_URL` — `https://abysscore-api.bassmit.dev/graphql`
- `NEXT_PUBLIC_KEYCLOAK_URL` — `https://abysscore-auth.bassmit.dev`
- `NEXT_PUBLIC_KEYCLOAK_REALM` — `abysscore`
- `NEXT_PUBLIC_KEYCLOAK_CLIENT_ID` — `abysscore-frontend`

3.4 Public Access via Cloudflare Tunnel

All five public endpoints route through a single `cloudflared` Deployment running two replicas. The tunnel is configured with a `config.yaml` that maps hostnames to internal Kubernetes services:

Hostname	Internal Service
`abysscore.bassmit.dev`	`frontend:3000`

`abysscore-api.bassmit.dev`	`gateway:4001`
`abysscore-auth.bassmit.dev`	`keycloak:8080`
`argocd.bassmit.dev`	`argocd-server:80`
`grafana.bassmit.dev`	`kube-prometheus-stack-grafana:80`

Cloudflare handles TLS for all public traffic. Inside the cluster, everything runs over plain HTTP — there's no internal certificate management to deal with.

3.5 Resource Management

Every Deployment has resource requests and limits defined. This serves two purposes: it lets Kubernetes schedule pods onto nodes correctly, and it makes Grafana's resource utilisation dashboards actually work (without requests, the "% of limit" panels show no data).

Observed resource usage from Grafana:

Service	CPU (typical)	Memory (typical)
Keycloak	variable	1.14 GiB
RabbitMQ	low	142 MiB
PostgreSQL	low	98 MiB
Cloudflared	low	22–27 MiB
Frontend	low	53 MiB
Backend	low	9–11 MiB
Gateway	low	8 MiB

Keycloak dominates memory usage. At current traffic levels, everything else is well within its limits.

3.6 Test and Production Environments

The current deployment runs a single environment on `main`. The logical next step is to split this into a proper test environment and a production environment, so that changes can be validated before they reach real users.

3.6.1 The Plan

The split follows a standard GitOps pattern: two branches, two namespaces, two sets of Cloudflare hostnames.

Environment	Branch	Namespace
Test	`develop`	`abysscore-test`
Production	`main`	`abysscore`

The `develop` branch is where all active work happens. Pull requests merge into `develop` first. Once a feature is tested and stable, it gets merged into `main` and goes live on production.

3.6.2 Image Tagging

GitHub Actions builds different image tags per environment:

- Push to `develop` → images tagged `:dev-<short-sha>` (e.g. `abysscore-backend:dev-a3f9c1`)
- Push to `main` → images tagged `:latest` and `:<sha>`

Using a unique tag per commit on `develop` means ArgoCD can detect new images reliably without relying on digest polling. Production keeps `:latest` for simplicity.

3.6.3 Kustomize Overlays

Rather than duplicating all manifests, the infra directory is restructured with Kustomize:

```
infra/k8s/
  base/          — shared manifests (Deployments, Services, ConfigMaps)
  overlays/
    test/         — patches: namespace, image tags, test hostnames
    prod/         — patches: namespace, prod hostnames, replicas
```

Each overlay patches only what differs between environments — the namespace, the image tag, and the Cloudflare hostnames. Everything else is inherited from `base/`.

3.6.4 ArgoCD Applications

Two ArgoCD Applications point at the two overlays:

- `abysscore-test` — watches the `develop` branch, applies `overlays/test/`, syncs to `abysscore-test` namespace
- `abysscore` — watches `main`, applies `overlays/prod/`, syncs to `abysscore` namespace (unchanged from current setup)

3.6.5 Cloudflare Hostnames

The test environment gets its own public URLs:

Hostname	Service
`test.abysscore.bassmit.dev`	`frontend (test)`
`test-api.abysscore.bassmit.dev`	`gateway (test)`
`test-auth.abysscore.bassmit.dev`	`keycloak (test)`

The tunnel config is extended to include these routes, all pointing at services in `abysscore-test`. Production hostnames remain unchanged.

3.6.6 Outcome

With this in place, the full workflow becomes:

1. Write code, push to `develop`
2. CI builds a `:dev-<sha>` image and pushes to GHCR
3. ArgoCD deploys it to the test environment automatically
4. Test at `test.abysscore.bassmit.dev`
5. Merge `develop` → `main` when ready
6. CI builds `:latest`, ArgoCD deploys to production

No manual steps, no risk of accidentally pushing broken code to prod.

Observability 4

4.1 The Monitoring Stack

Observability runs on kube-prometheus-stack, a Helm chart that bundles together Prometheus, Alertmanager, Grafana, kube-state-metrics, and node-exporter. It's the standard way to get monitoring set up on Kubernetes and it covers a lot of ground out of the box.

All monitoring images are mirrored to GHCR, same as the application images, because the Fontys proxy would block pulls from Quay and Docker Hub. The mirrored images are:

- `ghcr.io/bassmit12/prom-grafana`
- `ghcr.io/bassmit12/prom-prometheus`
- `ghcr.io/bassmit12/prom-operator`
- `ghcr.io/bassmit12/prom-alertmanager`
- `ghcr.io/bassmit12/prom-kube-state-metrics`
- `ghcr.io/bassmit12/prom-node-exporter`
- `ghcr.io/bassmit12/prom-config-reloader`
- `ghcr.io/bassmit12/prom-k8s-sidecar`

The full stack is managed by ArgoCD. `kube-prometheus-stack` pulls the Helm chart from prometheus-community and applies the values file from git. `monitoring-extras` applies ServiceMonitors and any additional configuration.

4.2 What Gets Scraped

Prometheus scrapes metrics from three places:

Kubernetes infrastructure — kube-state-metrics exposes metadata about every object in the cluster (Deployments, Pods, Services, etc.). Node-exporter runs on every node and exposes CPU, memory, disk, and network stats at the OS level. The Prometheus operator scrapes the API server, scheduler, and controller-manager — though on a managed cluster, the control plane components aren't exposed, so those targets stay down. That's expected.

The Prometheus operator itself — standard self-monitoring, useful for checking scrape health and TSDB status.

AbyssCore services — ServiceMonitors in the `abysscore` namespace define scrape targets for the frontend, backend, and gateway. Prometheus is configured with `serviceMonitorNamespaceSelector: {}` so it picks up ServiceMonitors from all namespaces, not just `monitoring`. The services have the correct labels (`app: abysscore-backend`, etc.) so the selectors match.

At the moment the backend and gateway return 404 on `/metrics` because neither has a Prometheus metrics endpoint implemented yet. The scrape targets exist and are being hit — Prometheus is just not collecting any application-level metrics yet. Node and cluster metrics are fully working.

4.3 Grafana

Grafana is accessible at `grafana.bassmit.dev` (admin password: configured in the values file). The default kube-prometheus-stack dashboards cover:

- **Kubernetes / Compute Resources / Namespace (Pods)** — CPU and memory usage per pod, with request and limit lines. This is the most useful day-to-day view.
- **Kubernetes / Compute Resources / Node** — node-level utilisation
- **Node Exporter / Nodes** — detailed OS-level metrics
- **Prometheus / Overview** — scrape health and storage stats

All AbyssCore pods show up in the namespace dashboard with live CPU and memory data. Resource limits are defined, so the utilisation percentage panels work correctly.

4.4 What's Missing

The current observability setup covers infrastructure well but has gaps at the application level:

No application metrics — the backend (Encore) can expose Prometheus metrics by adding `"metrics": {"type": "prometheus"}` to `infra.config.json`. The gateway can expose metrics with `promhttp.Handler()` at `/metrics`. Neither is done yet. Without application metrics, there's no visibility into request rates, error rates, latency, or game-specific events (dungeons created, combats resolved, etc.).

No custom dashboards — the default Grafana dashboards are useful but generic. An AbyssCore-specific dashboard showing game activity, API latency per service, and queue depth in RabbitMQ would be much more useful for understanding how the application actually behaves.

No alerting configured — Alertmanager is deployed but has no routing rules. Nothing sends a notification if a pod crashes or a service goes down.

No log aggregation — there's no Loki or similar stack. Logs are only accessible via `kubectl logs`, which works fine for debugging but means there's no historical log search or correlation with metrics.

Scalability 5

5.1 Current State

Right now, every service in AbyssCore runs as a single replica. There's one backend pod, one gateway pod, one frontend pod. If any of those goes down, that service is unavailable until Kubernetes restarts it. At current traffic levels this is fine — it's a personal project, not a production service — but it's worth understanding what "scaling" would actually look like here and what's available.

The one exception is `cloudflared`, which runs two replicas. That's intentional — if the single tunnel process crashes, all external traffic drops. Two replicas means there's always a fallback while Kubernetes restarts the failed one.

5.2 Load Balancing

Kubernetes does basic load balancing automatically. Every Service in Kubernetes is backed by `kube-proxy`, which distributes incoming connections across all healthy pods behind that Service using round-robin. If you scale a Deployment to three replicas, `kube-proxy` splits traffic roughly evenly across all three.

This means load balancing is essentially free — you just increase the replica count and Kubernetes handles distribution. There's no additional configuration needed. What you don't get out of the box is anything smarter than round-robin: no session affinity based on game state, no weighted routing, no circuit breaking. For those, you'd need a service mesh like Istio or Linkerd.

5.3 Horizontal Pod Autoscaler (HPA)

HPA is the standard Kubernetes mechanism for automatically scaling pod count based on resource utilisation. You define a target — say, "keep CPU usage below 70%" — and the HPA controller adds or removes pods to hit that target.

For AbyssCore, the services most likely to benefit from HPA are the backend and the gateway. They handle all the actual work: processing requests, running game logic, querying the database. The frontend is mostly static content and unlikely to become a bottleneck.

A reasonable HPA configuration for the backend would look like this:

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: abysscore-backend
  namespace: abysscore
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: abysscore-backend
  minReplicas: 1
```

```yaml
  maxReplicas: 5
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

This keeps at least one replica running, scales up to five if CPU hits 70%, and scales back down when load drops. Kubernetes applies a cooldown period between scale events to prevent flapping.

One important requirement: resource requests must be defined on the Deployment for HPA to work. HPA calculates utilisation as `actual_cpu / requested_cpu`. If there are no requests defined, the calculation fails and HPA can't do anything. AbyssCore already has resource requests defined on all Deployments, so HPA can be added without any other changes.

5.4 Vertical Pod Autoscaler (VPA)

VPA adjusts the CPU and memory requests and limits on a pod automatically, based on observed usage. Instead of adding more pods (like HPA), it makes each pod bigger or smaller.

VPA is useful when you don't know the right resource requests upfront, or when workload patterns change over time. In practice, it's less commonly used than HPA because it requires pod restarts to apply new values — VPA can't resize a running pod in place.

For AbyssCore, VPA could be useful for tuning the Keycloak memory allocation. At 1.14 GiB observed usage, the current limit is set generously. VPA would help find the right number automatically over time rather than guessing upfront.

VPA is not installed on the cluster currently. It would need to be deployed separately — it's not part of the standard Kubernetes distribution.

5.5 Cluster Autoscaler

The Cluster Autoscaler adds and removes nodes automatically based on whether pods can be scheduled. If all nodes are full and a pod is stuck in `Pending` state, the Cluster Autoscaler spins up a new node. When nodes are underutilised, it drains and removes them to save cost.

This is only relevant for cloud providers or infrastructure that supports dynamic node provisioning. The Fontys-managed cluster has a fixed set of nodes — `10.1.1.158` is the node AbyssCore runs on. There's no mechanism to add more nodes dynamically. The Cluster Autoscaler doesn't apply here.

5.6 Scaling Recommendations for Production

If AbyssCore were being prepared for real production traffic, here's what should change:

Add HPA for backend and gateway. These are the services that will see load under real usage. Setting a CPU target of 60–70% gives enough headroom to handle traffic spikes before things get slow.

Increase minimum replicas to 2. Running single replicas means any pod restart causes a brief outage. Two replicas eliminates this — Kubernetes can terminate and restart one pod while the other keeps serving traffic. This should apply to frontend, backend, and gateway at minimum.

Deploy VPA for Keycloak. Keycloak's memory usage is variable and hard to predict. VPA would tune the allocation automatically.

Separate PostgreSQL per service. Currently one Postgres instance handles all seven databases. Under load, this becomes a bottleneck and a single point of failure. The proper fix is a dedicated database instance per service, possibly with a connection pooler like PgBouncer in front.

Consider a service mesh for smart routing. For things like circuit breaking, retries with backoff, and traffic splitting during deployments, a lightweight service mesh would add significant value without much operational overhead.

None of this is urgent for the current state of the project, but it's the natural next step if AbyssCore were to grow beyond a personal project.

5.7 Planned: Authentication for All Services

Currently Keycloak only protects the frontend entry point. The gateway validates the JWT on every inbound request, but the individual backend services themselves don't verify the caller's identity — they trust that the gateway has already done it. That's fine as long as the gateway is the only path in, but it's a weak assumption. Any service reachable directly inside the cluster could be called without authentication.

The plan is to enforce authentication at every layer:

Gateway token validation. Already in place. Every request to the gateway must carry a valid JWT issued by the `abysscore` Keycloak realm. Requests without a token are rejected at the gateway before reaching any backend service.

Service-to-service auth. Each backend service should validate the JWT independently, rather than trusting the gateway implicitly. This can be done by passing the token through the gateway headers and having each Encore service verify the signature against the Keycloak JWKS endpoint. If a service is ever called directly — by mistake or by a misconfigured client — it will still reject the request.

Dedicated player login. Right now the only user is a single test account (`testplayer`). The plan is to open registration so anyone can create an account. Keycloak supports self-registration out of the box — enabling it in the realm settings is enough to get a working sign-up flow. Players would register with a username and password, Keycloak issues a JWT, and the existing NextAuth/Apollo flow handles the rest without any code changes.

Role-based access. Keycloak supports realm roles and client roles. Adding a `player` role to all registered users and an `admin` role for management operations would let the gateway enforce role-based access at the route level — `/admin/*` requires `admin`, all game endpoints require `player`. This keeps authorization logic out of the individual services and centralised at the gateway.

Together these changes would make the authentication boundary complete: Keycloak as the single source of truth for identity, the gateway as the enforcement point, and each service independently verifiable as a fallback.

References

Reflection and Future Work A

AA What Went Well

GitOps from the start. Setting up ArgoCD early meant that every change to the cluster went through git. This made it easy to track what changed and when, and made it impossible to end up with an undocumented cluster state. When something broke, there was always a clear history of what had changed.

Cloudflare Tunnel. This turned out to be a much cleaner solution than expected. It handles TLS, proxying, and DNS all in one. Adding a new public endpoint is just a one-line change to the tunnel config in git and a DNS record in Cloudflare. No cert-manager, no ingress controller, no LoadBalancer service — the Fontys network constraints actually pushed toward a simpler architecture.

Encore.go. The framework did a lot of heavy lifting. Database migrations, inter-service calls, routing, and API generation are all handled automatically. For a project with seven backend services, this would have been weeks of boilerplate otherwise.

Observability. Getting Prometheus and Grafana running with real data showing for all pods took some work — especially the image mirroring — but it's genuinely useful. Being able to see CPU and memory trends for every pod makes debugging much faster.

AB What Was Harder Than Expected

The Fontys TLS proxy. This wasn't mentioned anywhere in the documentation and took a while to diagnose. Every `ImagePullBackOff` error looked like a network issue, but the actual cause was the proxy intercepting the TLS connection to the image registry. Once the pattern was understood, the fix (mirror everything to GHCR) was straightforward, but it added friction to everything involving external images.

ArgoCD multi-source applications. The kube-prometheus-stack is installed from a Helm chart, but the values file lives in git. ArgoCD supports this as a multi-source Application but the configuration is not well documented and several attempts failed before finding the right format.

CI infinite loop from manifest updates. The `update-manifests` job commits back to `main` after every build. Without a guard, that commit triggers the workflow again — infinite loop. The fix is `paths-ignore: infra/k8s/**` on the workflow trigger, which is easy once you know it exists but took a few failed runs to diagnose.

GitHub Actions authentication for GHCR. The `GITHUB_TOKEN` that's built into GitHub Actions works for pushing to existing packages but fails with a 403 for new packages. This is a known limitation that's not prominently documented. The fix is a personal access token with `write:packages` scope stored as a secret (`GHCR_TOKEN`).

AC What Changed from the Project Plan

The original project plan proposed an AI-powered monitoring system using Cilium and eBPF for anomaly detection in Kubernetes infrastructure. The shift to AbyssCore changed the focus from infrastructure research to application development and deployment.

The core skills are similar — Kubernetes, observability, GitOps — but the approach changed from studying existing tools to building and operating a real system. The result is more practical and more hands-on than the original plan, and covers more of the actual engineering challenges that come up when running real software on Kubernetes.

AD Future Work

Application metrics. The backend and gateway don't expose `/metrics` endpoints yet. Adding Prometheus instrumentation to both would give visibility into request rates, latency, error rates, and game-specific counters. For Encore, this is a one-line config change.

Custom Grafana dashboard. A dashboard showing AbyssCore-specific data — active players, dungeons in progress, combat outcomes, queue depth — would make the observability much more meaningful.

Alerting. Alertmanager is deployed but not configured. Basic alerts for pod restarts, high memory usage, and service unavailability would make the setup more production-like.

Log aggregation. Loki would add log search and correlation with metrics. Currently logs are only available via `kubectl logs`, which has no retention and no search.

HPA for backend and gateway. As described in the scalability chapter, adding Horizontal Pod Autoscalers to both services is a natural next step and is straightforward to implement.

Image Updater. ArgoCD Image Updater can automatically update Deployment manifests when a new image is pushed to GHCR. This would close the loop on CI/CD — push code, image gets built and pushed, ArgoCD detects the new digest and rolls it out automatically.

Appendix A — Kubernetes Manifest Overview

B

All manifests live under `infra/k8s/` in the AbyssCore repository. The structure is flat — all core application manifests are in `infra/k8s/` and monitoring extras are in `infra/k8s/monitoring/`.

File	Contents
`backend.yaml`	Deployment + Service for the Encore backend
`frontend.yaml`	Deployment + Service for the Next.js frontend
`gateway.yaml`	Deployment + Service for the GraphQL gateway
`cloudflared.yaml`	Deployment for Cloudflare Tunnel (2 replicas)
`keycloak.yaml`	Deployment + Service for Keycloak
`rabbitmq.yaml`	Deployment + Service for RabbitMQ
`postgres.yaml`	StatefulSet + Service for PostgreSQL
`secrets.yaml`	Kubernetes Secrets (GHCR pull secret, DB creds, etc.)
`monitoring/kube-prometheus-values.yaml`	Helm values for kube-prometheus-stack
`monitoring/service-monitors.yaml`	ServiceMonitors for backend, gateway, frontend

Appendix B — ArgoCD Application Definitions

C

ArgoCD Applications are stored in `infra/argocd/`:

File	Manages
`abysscore-app.yaml`	All game services from `infra/k8s/`
`monitoring-app.yaml`	kube-prometheus-stack Helm chart
`monitoring-extras-app.yaml`	ServiceMonitors from `infra/k8s/monitoring/`

Appendix C — CI/CD Pipeline Summary

D

The GitHub Actions workflow (`.github/workflows/build-push.yml`) runs three parallel build jobs on every push to `main` (excluding changes to `infra/k8s/`):

Job	Registry Target	Build Tool
Build Backend	`ghcr.io/bassmit12/abysscore-backend:<sha>`	`encore build docker`
Build Gateway	`ghcr.io/bassmit12/abysscore-gateway:<sha>`	Docker Buildx
Build Frontend	`ghcr.io/bassmit12/abysscore-frontend:<sha>`	Docker Buildx

All three images are also tagged `:latest`. After the build jobs, an `update-manifests` job patches the SHA tag in each `infra/k8s/*.yaml` and commits the change back to `main`. ArgoCD detects the updated manifest and rolls out the new images automatically.

The `paths-ignore: infra/k8s/**` filter on the workflow trigger prevents the manifest-update commit from triggering another build run.

Authentication uses the `GHCR_TOKEN` secret (PAT with `write:packages` scope). The `GITHUB_TOKEN` built into Actions fails with 403 for new packages.